COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	06/10/2026	**DATE SUBMITTED:**	06/10/2026
NAME:	Payas, John Paul S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A	
A.1. Create a Graph	**CODE**:

```cpp
#include <iostream>
#include <vector>
#include <utility>

class Graph {
private:
    std::vector<int> vertices;
    std::vector<std::pair<int, int>> edges;

public:
    // Add a vertex
    void addVertex(int vertex) {
        vertices.push_back(vertex);
    }

    // Add an edge
    void addEdge(int vertex1, int vertex2) {
        edges.push_back({vertex1, vertex2});
    }

    // Display the graph
    void displayGraph() {
        std::cout << "Vertices: ";

        for (int vertex : vertices) {
            std::cout << vertex << " ";
        }

        std::cout << "\nEdges:\n";

        for (const auto& edge : edges) {
            std::cout << edge.first << " - " << edge.second << std::endl;
        }
    }
};
```

```
int main() {
    Graph graph;

    // Add vertices
    graph.addVertex(1);
    graph.addVertex(2);
    graph.addVertex(3);
    graph.addVertex(4);
    graph.addVertex(5);

    // Add edges
    graph.addEdge(1, 2);
    graph.addEdge(1, 3);
    graph.addEdge(1, 4);
    graph.addEdge(2, 5);
    graph.addEdge(3, 4);
    graph.addEdge(4, 5);

    // Display graph
    graph.displayGraph();

    return 0;
}
```

OUTPUT:

```
Vertices: 1 2 3 4 5
Edges:
1 - 2
1 - 3
1 - 4
2 - 5
3 - 4
4 - 5
```

A.2. Adjacency Matrix

CODE:

```cpp
#include <iostream>

int main() {
    char vertices[] = {'A', 'B', 'C', 'D', 'E'};

    int matrix[5][5] = {
        {0, 1, 1, 1, 0},
        {1, 0, 0, 0, 1},
        {1, 0, 0, 1, 0},
        {1, 0, 1, 0, 1},
        {0, 1, 0, 1, 0}
    };

    std::cout << "Adjacency Matrix:\n\n";

    std::cout << "  ";
    for (int i = 0; i < 5; i++) {
        std::cout << vertices[i] << " ";
    }

    std::cout << std::endl;

    for (int i = 0; i < 5; i++) {
        std::cout << vertices[i] << " ";

        for (int j = 0; j < 5; j++) {
            std::cout << matrix[i][j] << " ";
        }

        std::cout << std::endl;
    }

    return 0;
}
```

```cpp
#include <iostream>

int main() {
    char vertices[] = {'A', 'B', 'C', 'D', 'E'};

    int matrix[5][5] = {
        {0, 4, 3, 0, 0},
        {0, 0, 0, 0, 0},
        {0, 0, 0, 2, 0},
        {0, 1, 0, 0, 1},
        {1, 0, 0, 0, 0}
    };

    std::cout << "Weighted Directed Adjacency Matrix:\n\n";

    std::cout << "  ";
    for (int i = 0; i < 5; i++) {
        std::cout << vertices[i] << " ";
    }

    std::cout << std::endl;

    for (int i = 0; i < 5; i++) {
        std::cout << vertices[i] << " ";

        for (int j = 0; j < 5; j++) {
            std::cout << matrix[i][j] << " ";
        }

        std::cout << std::endl;
    }

    return 0;
}
```

OUTPUT:

```
Adjacency Matrix:

  A B C D E
A 0 1 1 1 0
B 1 0 0 0 1
C 1 0 0 1 0
D 1 0 1 0 1
E 0 1 0 1 0
```

```
Weighted Directed Adjacency Matrix:

  A B C D E
A 0 4 3 0 0
B 0 0 0 0 0
C 0 0 0 2 0
D 0 1 0 0 1
E 1 0 0 0 0
```

A.3. Adjacency List	CODE:

```cpp
#include <iostream>

struct adjNode {
    int val;
    int cost;
    adjNode* next;
};

struct graphEdge {
    int start_ver;
    int end_ver;
    int weight;
};

class DiaGraph {
private:
    int N;

    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;

        return newNode;
    }

public:
    adjNode** head;

    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;

        head = new adjNode*[N];
```

```
    for (int i = 0; i < N; i++) {
        head[i] = nullptr;
    }

    for (int i = 0; i < n; i++) {
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;

        adjNode* newNode =
            getAdjListNode(end_ver, weight, head[start_ver]);

        head[start_ver] = newNode;
    }
}

~DiaGraph() {
    for (int i = 0; i < N; i++) {
        adjNode* current = head[i];

        while (current != nullptr) {
            adjNode* temp = current;
            current = current->next;
            delete temp;
        }
    }

    delete[] head;
}
```

```
    void display_AdjList(adjNode* ptr, int i) {
        while (ptr != nullptr) {
            std::cout << "(" << i << ", "
                      << ptr->val << ", "
                      << ptr->cost << ") ";

            ptr = ptr->next;
        }

        std::cout << std::endl;
    }
};

int main() {
    graphEdge edges[] = {
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6;

    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph diagraph(edges, n, N);

    std::cout << "Graph adjacency list\n";
    std::cout << "(start_vertex, end_vertex, weight):\n";
```

```
    for (int i = 0; i < N; i++) {
        diagraph.display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

OUTPUT:

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Vertex	Adjacency List (Start, End, Weight)
0	(0, 2, 4) (0, 1, 2)
1	(1, 4, 3)
2	(2, 3, 2)
3	(3, 1, 4)
4	(4, 3, 3)
5	No connection

ILO B	
B.1 Depth-First Search (DFS)	**CODE:**

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    std::vector<Edge<T>> edges() const {
        return edge_list;
    }

    void add_edge(const Edge<T>& edge) {
```

```
        edge_list.push_back(edge);
    }

    std::vector<Edge<T>> outgoing_edges(size_t vertex) const
        std::vector<Edge<T>> result;

        for (const auto& edge : edge_list) {
            if (edge.src == vertex) {
                result.push_back(edge);
            }
        }

        return result;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
```

```
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
```

```
    for (auto& i : edges) {
        for (auto& j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}

int main()
{
    auto G = create_reference_graph<unsigned>();

    std::cout << "DFS Order of vertices:\n";

    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order) {
        std::cout << v << std::endl;
    }

    return 0;
}
```

OUTPUT:

```
DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

CODE:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    void add_edge(const Edge<T>& edge) {
        if (edge.src >= V || edge.dest >= V) {
            return;
        }
```

```
        edge_list.push_back(edge);
    }

    std::vector<Edge<T>> outgoing_edges(size_t vertex) const {
        std::vector<Edge<T>> result;

        for (const auto& edge : edge_list) {
            if (edge.src == vertex) {
                result.push_back(edge);
            }
        }

        return result;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
```

```
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                if (visited.find(e.dest) == visited.end()) {
                    queue.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
```

```
    for (auto& i : edges) {
        for (auto& j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}

int main()
{
    auto G = create_reference_graph<unsigned>();

    std::cout << "BFS Order of vertices:\n";

    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order) {
        std::cout << v << std::endl;
    }

    return 0;
}
```

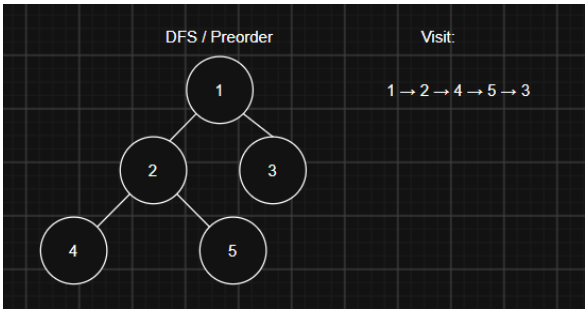
OUTPUT:

```
BFS Order of vertices:
1
2
5
4
8
3
6
7
```

TABLE:

Algorithm	Output
DFS	1 → 5 → 8 → 6 → 7 → 3 → 4 → 2
BFS	1 → 2 → 5 → 4 → 8 → 3 → 6 → 7

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C	
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.	- DFS is most helpful because it follows one path as deeply as possible before backtracking to explore other vertices. This matches the situation where a person visits one location, backtracks, and then explores another location from the same vertex.
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.	- DFS in a tree is commonly represented by preorder traversal because it visits a node before recursively exploring its children. Both approaches follow a depth-first strategy and use backtracking when there are no more unvisited nodes. 
3. In the performed code, what data structure is used to implement the Breadth First Search?	- BFS uses a queue to store vertices waiting to be visited. The first vertex placed in the queue is the first one removed, allowing BFS to process the graph level by level. The activity's implementation explicitly declares
4. How many times can a node be visited in the BFS?	- A node should be marked as visited when it is processed so that it is not counted or processed again. The activity uses a std::set named visited to prevent repeated visits.

CONCLUSION

Summary of Lessons Learned

I learned how graphs can be represented using adjacency matrices and adjacency lists. I also learned how DFS and BFS traverse vertices using stacks and queues.

Analysis of the Procedure

The procedure helped me understand how vertices and edges are represented and how graph traversal works. Implementing the programs made it easier to understand the difference between DFS and BFS.

Analysis of the Supplementary Activity

The supplementary activity showed that DFS is useful for depth-based exploration while BFS is useful for level-by-level traversal. It also helped me understand the importance of stacks, queues, and visited-node tracking.

Concluding Statement / Feedback

I was able to do the basic implementation and traversal of graphs through the activity. I still need to improve my understanding of graph representations and become more familiar with implementing traversal algorithms

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It just fixed my grammar but it was my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.